Particles in Underworld3

Louis Moresi[1]

[1] Australian National University

DOI https://doi.org/10.6084/m9.figshare.33193599

Abstract

Underworld is built around the idea of active Lagrangian tracer particles that carry history and composition information as the material deforms. How do we combine this information with our symbolic mathematical framework ?

Keywords Underworld Code, Documentation, development

PUBLISHED Jun 03, 2026
ARTICLE UWTN 2026-008
VERSION 1.0.0

ARCHIVED 2026-08-10T04:19:43Z
SOURCE https://www.underworldcode.org/particles-in-underworld3/

Lagrangian particles in a fixed-mesh, finite-element code can be used to carry material properties with the flow. Composition, strain history, stress memory, damage. Finite element solvers operate on fields defined on the mesh, so scattered data on particles always demand special treatment of some kind. In Underworld 1 and Underworld 2, we created dynamic integration schemes for particle swarms on an element-by-element basis. This option is not available with the PETSc point-wise function approach that UW3 uses. For this, we need to project particle data onto the available interpolating functions before each solve.

We also need to be able to represent the particle-based data and its derivatives symbolically for the underworld3 representation to be composable with mesh-based data when we construct the weak form. In Underworld3, we create `swarmVariables` as natural, symbolic objects to fulfil this requirement.

A swarm variable has a `.sym` property that returns a SymPy symbol, just like a mesh variable. That symbol participates in the solver's weak form, the constitutive model, the boundary conditions. The solver does not know whether it is reading a field computed on the mesh or a field projected from particles. The distinction is invisible at the symbolic level.

1. The Problem

A Stokes solver assembles the weak form by evaluating expressions at quadrature points inside each element. The expressions reference field variables defined at mesh nodes. If the viscosity depends on a material property stored on particles, that property must first be available as a mesh field.

In Underworld2, the user managed this explicitly. You would call a projection routine before each solve, mapping particle data onto the mesh. Forget the projection, and the solver would use stale data.

UW3 automates this through a ***"proxy mesh variable"*** pattern.

2. Swarm Variables

Particles carry data on swarm variables which are analogous to mesh variables and have almost identical usage patterns:

```
swarm = uw.swarm.Swarm(mesh)
swarm.populate(fill_param=3)

material_C = uw.swarm.SwarmVariable("material_C", swarm, size=1)
material_C.array[:] = initial_values
```

Each swarm variable is stored as a PETSc field on a `DMSwarm`. When particles migrate between processors, their variable data travels with them automatically. The `.array` property provides user-facing access to the underlying storage, with the same NDArray_With_Callback mechanism described in the arrays-in-sync post. Underneath `.array` sits the `.data` property, which exposes the raw non-dimensional values PETSc operates on directly.

A swarm variable can be scalar, vector, or tensor, or an arbitrary matrix shape:

```
stress_history = uw.swarm.SwarmVariable(
    "stress", swarm, vtype=uw.VarType.SYM_TENSOR, proxy_degree=2
)
```

The `proxy_degree` parameter is optional and sets the polynomial degree of the companion mesh variable that UW3 uses to project particle data onto the mesh. It defaults to `1` (linear interpolation); higher values give a smoother proxy at the cost of more mesh degrees of freedom. Every swarm variable gets a proxy by default: you do not have to ask for one, but you can ask *not* to proxy the variable (for example, it makes no sense to proxy a discrete-index variable)

3. The Proxy Mesh Variable

The companion mesh variable is a standard finite element field defined on the mesh nodes, at the polynomial degree given by `proxy_degree`.

The projection from particles to mesh uses inverse-distance-weighted (IDW) interpolation from the nearest particles to each mesh node, implemented through a KDTree of particle positions:

1. Build a KDTree of all particle positions on the local rank.
2. For each mesh node at position x_n, find the k nearest particles ($k = \text{dim} + 1$ by default), giving neighbour positions $x_p^{(i)}$ and values $\phi_p^{(i)}$ for $i = 1, \ldots, k$.
3. Compute weights from the squared distances $d_i^2 = | \, x_n - x_p^{(i)} \, |^2$ and take the normalised weighted average:

$$w_i = \frac{1}{(\varepsilon + d_i^2)^p}, \qquad \phi_n = \frac{\sum_{i=1}^{k} w_i \, \phi_p^{(i)}}{\sum_{i=1}^{k} w_i} \tag{1}$$

The default exponent is $p = 2$, giving weights that decay as $1/d^4$. The small ε regularises the case where a particle coincides with a mesh node — this happens more often than you might imagine in practice, because we often use swarms to carry mesh information during deformation or during advection.

1. Store the result ϕ_n on the proxy mesh variable.

The proxy has a `.sym` property that returns a SymPy symbol. This is the same interface as any mesh variable. You use it in expressions, constitutive models, boundary conditions, source terms. The solver sees a mesh variable symbol and does not need to know that the data originated on particles.

4. Lazy Evaluation

The projection is not free. Building a KDTree, querying nearest neighbours, computing weights, and writing to the mesh variable all take time. Doing this before every solver assembly would be wasteful when the particle data has not changed.

UW3 handles this through lazy evaluation. The swarm variable tracks whether its data has been modified since the last projection. When you access `.sym`, the system checks this flag. If the data is stale, it triggers a fresh projection. If not, it returns the cached proxy symbol immediately.

```
# Modify particle data
material_C.array[some_particles] = new_values   # marks proxy as stale

# Next access to .sym triggers projection
viscosity_fn = eta_0 * sympy.exp(-material_C.sym)   # projection happens here
```

During solver assembly, the same symbol may be evaluated many times at different quadrature points. The projection happens once, on the first access after a modification. Subsequent accesses within the same solve use the cached mesh field.

This means the user never calls a projection routine. Modify particle data, use the symbol, the framework does the rest.

5. Particles in Expressions

Because `.sym` returns a standard SymPy symbol, particle data composes with everything else in UW3's symbolic layer:

```
# material_C is a swarm variable (per-particle);
# Temp is a mesh variable (per-node).
viscosity_fn = eta_0 * sympy.exp(-material_C.sym * Temp.sym)

# Use in constitutive model
stokes.constitutive_model.Parameters.shear_viscosity_0 = viscosity_fn
```

```
# The solver differentiates through this for the Jacobian
# It does not know that material_C.sym comes from particles
```

The same pattern works for stress history in viscoelastic models. The DFDt infrastructure stores previous stress values on a swarm variable like the `stress_history` declared above, and the constitutive model reads its `.sym` as part of the effective strain-rate expression. The projection from particles to mesh is lazy, the symbol is SymPy, and the Jacobian derivation is automatic.

This is the same design principle that applies throughout UW3: separate the physics from the numerics through symbolic expressions. For constitutive models, the boundary is the stress tensor. For time derivatives, the boundary is the BDF/AM expression. For particles, the boundary is the proxy mesh variable symbol.

6. Batching Updates

Migration involves MPI communication and KDTree reconstruction. If you are updating particle coordinates and modifying variable data in the same operation, you do not want to trigger migration and projection after each step. The `uw.synchronised_array_update()` context manager batches changes and defers the synchronisation to the end of the block:

```
with uw.synchronised_array_update():
    swarm.particle_coordinates.array[...] = new_coords
    material_C.array[:] = new_values
# Migration and cache invalidation happen here, once
```

The same context manager works for mesh variables — batch any combination of mesh and swarm variable writes, and let the framework handle the communication once at the end.

7. What the Solver Sees

From the solver's perspective, there are only mesh variable symbols. The JIT compiler generates C code that reads field values at quadrature points. Whether those values were computed by the solver, set by the user, or projected from particles is an implementation detail behind the symbol.

This uniformity is important. It means you can start with a mesh-based viscosity field, decide later to track viscosity on particles for better advection properties, and the solver code does not change. You swap the symbol from a mesh variable to a swarm variable proxy. The constitutive model, the weak form, the Jacobian, the compiled C all remain the same.

The Underworld project is supported by AuScope and the Australian Government through the National Collaborative Research Infrastructure Strategy (NCRIS). Source code: github.com/underworldcode/underworld3